

[BLUE PULSE](#) ► [DOCS](#)

VulnFlow · Documentación técnica y de funcionamiento

Documentación técnica y de funcionamiento

[Python](#)[FastAPI](#)[PostgreSQL 16 / SQLite](#)[Ed25519](#)[Jinja2](#)

VulnFlow · Documentación técnica y de funcionamiento

Documento de referencia para entender **cómo funciona** VulnFlow por dentro y **dónde vive cada pieza**. Lectura recomendada en este orden: visión general → el flujo de una auditoría → el núcleo (`core/`) → la API → el frontend → la infraestructura → seguridad → cómo extenderlo.

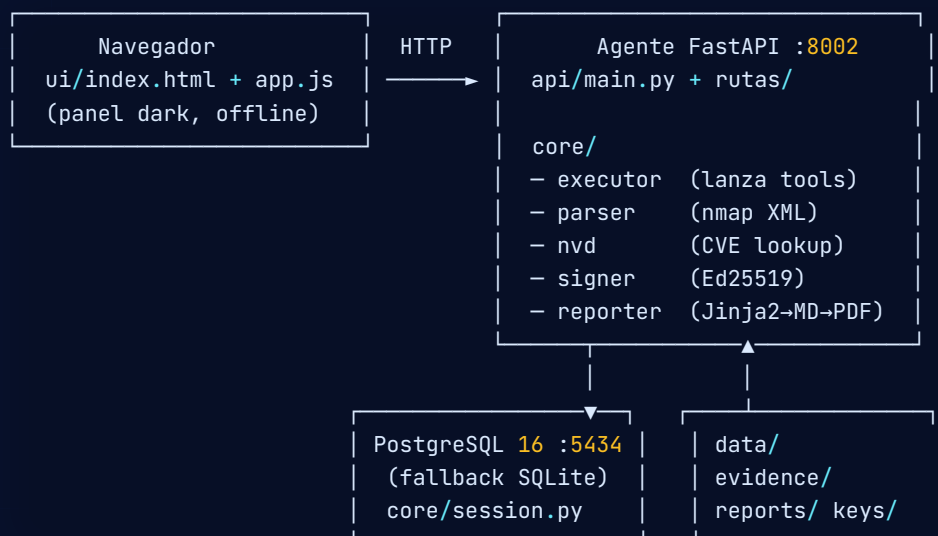
1. Visión general

VulnFlow es un **orquestador de pentesting**: no implementa ningún scanner, sino que lanza las herramientas ya instaladas en el sistema (nmap, gobuster, ffuf, dig, curl, metasploit) contra un activo autorizado, **captura la salida completa** de cada ejecución y permite al auditor **marcar** cualquier fragmento de esa salida como **hallazgo** (con severidad, remediación y CVEs opcionales). Con todo eso genera un **informe técnico profesional en Markdown**, lo exporta a **PDF** y lo **firma con Ed25519** para garantizar integridad y autenticidad.

Tres decisiones de diseño que lo definen:

1. **Eres tú quien decide qué es un hallazgo.** La herramienta no puntúa automáticamente nada; conserva la salida en crudo y vos marcás.
2. **Todo comando queda registrado.** Cada ejecución guarda el argv exacto y su salida en `data/evidence/`. Nada se pierde, nada se inventa.
3. **El informe se genera solo.** Se construye siempre sobre lo que hay en la BD (activos → ejecuciones → hallazgos → evidencias) en el momento de generarlo.

2. Arquitectura general



- **Persistencia:** SQLAlchemy 2 sobre **PostgreSQL 16** (Docker, host :5434) con **degradación automática a SQLite** si el contenedor no responde.
- **Ejecución de herramientas:** `subprocess.Popen` con **argv en lista** (jamás `shell=True`), proceso en su propio grupo (para matarlo entero en un timeout).
- **Frontend:** HTML/CSS/JS vanilla, servido por el propio FastAPI, **100% offline** (tipografías Inter + JetBrains Mono vendidas localmente).

Cadena de confianza del informe

Hallazgo ← ejecución (comando exacto) ← activo objetivo ← evidencia en crudo ← hash firmado

El `*.sig` (Ed25519) cubre los **bytes finales** del Markdown, que incluye la línea con el SHA-256 del propio documento: cualquier alteración rompe la firma.

3. El flujo de una auditoría (paso a paso)

1. **Registro del activo** — `POST /targets` con IP/dominio (+ hostname, SO, descripción). Se valida con `v_host` (IP o dominio bien formado) → fila en `targets`.
2. **Ejecución de un módulo** — `POST /scans { target_id, module_slug, params }`.
3. Se busca el `Module` en el catálogo y se comprueba la herramienta en el PATH (`shutil.which`); si falta → `ToolUnavailable` → `400`.
4. `build_argv()` valida/normliza los parámetros contra el esquema del módulo y devuelve la **lista argv**.
5. Se persiste un `ToolRun` con `status=running` y el comando como `shell-join`, y se hace `commit` (así el run existe aunque la herramienta falle).
6. Se lanza con `start_new_session=True` y timeout (por defecto 300 s). Si expira: `killpg(SIGTERM)` → `killpg(SIGKILL)` si sigue vivo; `status=timeout`.

7. Si termina: `status=ok|error` según `returncode`.
8. La salida completa (`stdout+stderr`) se escribe en `data/evidence/run_<id>/output.txt` ; si el primer KB contiene `<nmaprun` , además en `output.xml` (para el parser). Se actualiza `end_time`.
9. **Marcado de un hallazgo** — en el panel, seleccionas texto de la salida y pulsas « Marcar selección». La API `POST /findings` :
10. valida severidad (`LOW/MEDIUM/HIGH/CRITICAL`) y título;
11. crea `Finding` (vinculado al `run_id`);
12. persiste la evidencia en `data/evidence/finding_<id>.txt` y crea su fila `Evidence` ;
13. si `lookup_cves` , consulta la NVD y guarda el CSV en `Finding.cve_ids` .
14. **Generación del reporte** — `POST /reports` :
15. `core/reporter.generate_report()` construye el contexto completo de la BD y lo renderiza con Jinja2 (`templates/report_template.md.j2`);
16. opcionalmente **enriquece CVEs**: parsea el XML de los runs nmap y consulta la NVD por cada (producto, versión) detectado;
17. opcionalmente `to_pdf()` convierte el Markdown a PDF (WeasyPrint);
18. opcionalmente `sign_report()` incrusta el SHA-256 en el documento y firma.
19. **Verificación** — `GET /reports/<file>/verify` (o `.venv/bin/python verify.py`) comprueba la firma contra `data/keys/signer_ed25519.pub` .

4. El núcleo: `core/`

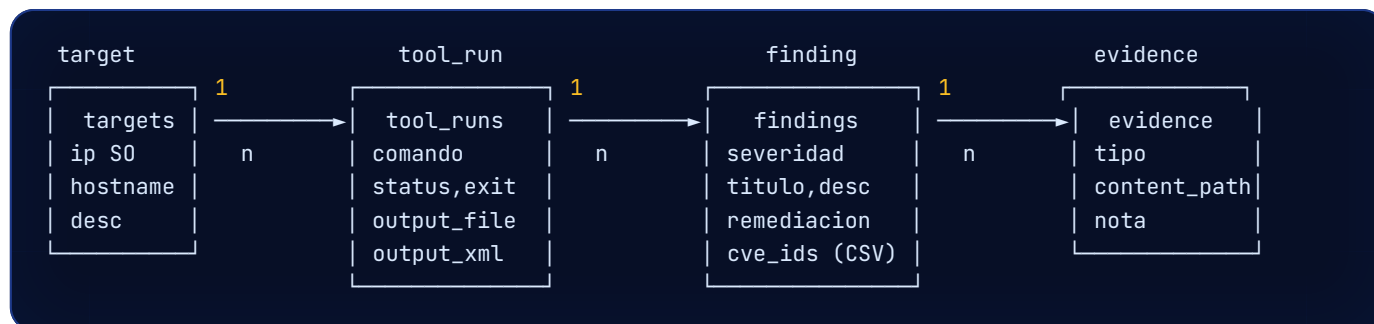
4.1 `config.py` — configuración central

Variable	Defecto	Sentido
<code>DATABASE_URL</code>	<code>postgresql+psycopg://vulnflow:vulnflow@127.0.0.1:5434/vulnflow</code>	DSN principal
<code>VULNFLOW_HOST</code>	<code>127.0.0.1</code>	bind de uvicorn
<code>VULNFLOW_PORT</code>	<code>8002</code>	puerto de la API
<code>VULNFLOW_TIMEOUT</code>	<code>300</code>	segundos máximos por run
<code>VULNFLOW_ALLOW_RAW</code>	<code>0</code>	habilita el módulo <code>raw</code> (msfconsole)

Los directorios de datos (`data/{evidence,reports,keys}`) se crean al importar. `dump()` expone un resumen sanitizado (oculta la contraseña del DSN) usado en `/health` y en los logs de arranque.

4.2 `models.py` — el modelo de datos

Relación central:



- **Target** — el activo. `cascade="all, delete-orphan"` en `runs` (borrar activo destruye su historial completo).
- **ToolRun** — una ejecución. `command_executed` es el argv reconstruido con `shlex.join` (para el informe y la cita del hallazgo). `output_file` / `output_xml` son **rutas relativas** a `data/evidence/`. Estado: `running|ok|timeout|error`.
- **Finding** — el hallazgo. `severity` es un **Enum de PostgreSQL** estricto (`LOW/MEDIUM/HIGH/CRITICAL`): conviene normalizar a `.upper()` antes de insertar (SQLite lo tolera; el enum de Postgres no).
- **Evidence** — el material: por ahora `type="text"` apuntando al `.txt` en el disco; el modelo ya prevé `image|file`.

Siempre se usan `datetime UTC` con zona horaria (`DateTime(timezone=True)`) y timestamps auto-asignados.

4.3 `session.py` — persistencia con degradación elegante

`get_engine()` devuelve un engine cacheado (una sola creación):

1. DSN `sqlite://` explícito → SQLite tal cual.
2. DSN PostgreSQL → **sonda** (`SELECT 1` con `connect_timeout=2` y `frobenius pool_pre_ping=True`). Si responde → PostgreSQL.
3. Si no responde → SQLite `data/vulnflow.db` y aviso por log.

Consecuencias prácticas:

- La API **arranca siempre**, da igual si el contenedor de la BD está levantado o no.
- El engine queda fijado a la primera conexión: **si cambias `DATABASE_URL`, reinicia el agente**.
- `init_db()` crea las tablas (idempotente) vía `Base.metadata.create_all()`.
- `session_scope()` entrega sesiones (`expire_on_commit=False`).

4.4 `executor.py` — el motor de ejecución (núcleo duro)

Abstracciones:

- `Param(name, label, type, required, default, hint, validate)` — esquema de un parámetro con su validador opcional.
- `Module(slug, name, tool, category, description, params, raw, builder)` — una herramienta del catálogo. `builder(target, params) → list[str]` fabrica el argv.
- **MODULES** — el catálogo (8 módulos; ver tabla §4.5). `MODULES_BY_SLUG` permite búsqueda O(1). `module_public()` expone la vista no ejecutable para la API/UI.

Validadores (`v_*`) usados por los parámetros:

Validador	Acepta	Rechaza
v_host	IP (v4/v6) o hostname <code>[A-Za-z0-9][A-Za-z0-9._-]{0,253}</code>	cadena con espacios / símbolos
v_port_range	<code>80 / 1-1000 / 80,443,8000-8100</code>	letras, rangos malformados
v_int	dígitos	—
v_wordlist	ruta <code>.txt</code> alfanumérica con <code>/_.-</code>	rutas con espacios/ <code> / ;</code>
v_url	<code>http://</code> o <code>https://</code>	cualquier otra cosa

`_validate_params()` normaliza los valores (string) contra el esquema y **descarta parámetros extra** (seguridad por diseño: el cliente no puede inyectar `--ports`).

Ejecución (`execute`) — el contrato completo:

```

module_slug inválido      → ValidationError → 400
tool ausente en PATH      → ToolUnavailable → 400
parámetros inválidos     → ValidationError → 400
módulo raw sin ALLOW_RAW → ValidationError → 400
proceso no arranca       → RuntimeError (status=error ya persistido)
timeout                  → status=timeout (grupo matado en bloque)
fin normal               → status=ok|error según returncode

```

`_write_outputs()` además **autodetecta XML de nmap** en el primer KB y lo guarda aparte (`output.xml`), que es lo que consume `core/parser.py`.

Por qué sin shell: los parámetros nunca tocan un intérprete. El argv va directo a `Popen`. No existe un camino en el que una cadena del cliente pueda convertirse en `;`, `|`, `$()` o backtick.

4.5 El catálogo de módulos

slug	Herramienta	builder	Parámetros
nmap_ports	nmap	<code>_build_nmap("ports")</code>	<code>ports</code> (port_range, opcional)
nmap_services	nmap	<code>_build_nmap("services")</code>	<code>ports</code>
nmap_vuln	nmap	<code>_build_nmap("vuln")</code>	—
gobuster_dir	gobuster	<code>_build_gobuster</code>	<code>url</code> , <code>wordlist *</code> , <code>status_codes</code>
ffuf_vhost	ffuf	<code>_build_ffuf</code>	<code>url *</code> (con <code>FUZZ</code>), <code>wordlist *</code> , <code>match_codes</code>
dig_dns	dig	<code>_build_dig</code>	<code>record</code> (A/A/AAAA/MX/NS/TXT/CNAME)
curl_http	curl	<code>_build_curl</code>	<code>url</code> , <code>timeout</code>
msfconsole	msfconsole	<code>_build_msf</code>	<code>script *</code> — <code>raw=True</code> (Requiere <code>VULNFLOW_ALLOW_RAW=1</code>)

Los builders de nmap fijan `-sT -Pn` siempre (TCP connect, compatible con el agente), `-sV` para services/vuln, `-oX -` para emitir XML a stdout. `_build_msf` y los otros nunca interpolan cadenas del usuario fuera del argv.

4.6 parser.py — limpieza de salida

- `parse_nmap_xml(raw)` → estructura defensiva `[ {ip, hostname, os, ports:[...]} ]`, solo puertos **open**, con `service/product/version/cpe`. **Nunca lanza**: XML roto → `[]`.
- `parse_open_ports_from_text(raw)` — fallback barato: regex sobre salida de texto.
- `extract_version_hints(target)` → (producto, versión) para alimentar la NVD.

4.7 nvd.py — sugerencia de CVEs (bonus con degradación limpia)

Cliente REST v2 de la NVD:

- Cache por (keyword, limit) en memoria.
- Rate-limit de cortesía (≥ 6 s entre llamadas reales).
- `search_cves(keyword, limit)` → `[{cve_id, description, base_score, published}]`.
- `suggest_for_version(product, version)` — comodidad para el enriquecimiento.
- **Cualquier error de red/API** → `[]`, nunca una excepción al flujo de auditoría.
- `base_score` sale de `cvssMetricV31` (o V30) si está presente.

4.8 signer.py — firma Ed25519

- Clave privada `data/keys/signer_ed25519.key` (PEM PKCS8, `chmod 0600`) generada a la primera; pública a `signer_ed25519.pub`.
- `sign_file(path)` → `path.sig` junto al original, con el SHA-256 (base64 y hex) y la firma base64 + clav pública en una sola línea.
- `verify_file(path)` → `{ok, sha256_hex}`; cualquier problema (firma inválida, `.sig` ausente, formato raro) → `{ok:false, error}`.
- Funciona igual sobre Markdown que sobre cualquier otro archivo.

4.9 reporter.py — del estado de la BD al informe

- `ReportOptions`: `target_ids` (None = todos), `enrich_cves`, `filename`.
- `_build_context()` recorre `Target` → `runs` → `findings` → `evidence` y calcula:
- resumen por severidad (`summary`),
- por finding: `ref = VF-0000`, `cves` (lista), `evidence_text` (concatena los ficheros de texto de sus evidencias), `evidence_files`,
- por run: comando, duración, estado y `output_excerpt` (**máx. 4000 caracteres**, para que el informe no pese al infinito),
- enriquecimiento CVE (si se pide): parsea XML nmap de cada run y consulta la NVD con caché por (producto, versión).
- `generate_report()` → Jinja2 (`Environment` con `FileSystemLoader`, sin autoescape porque es Markdown) → `data/reports/report_<prefix>_<YYYYMMDD-HHMMSS>.md`.
- `sign_report()`: **primero sustituye el SHA-256 vacío** que dejó la plantilla (el hash se calcula sobre el texto sin incrustar), **después** `sign_file()` firma los bytes finales. Así el `.sig` cubre exactamente el contenido guardado.
- `to_pdf()`: Markdown→HTML (`markdown` lib, tablas/fenced code/nl2br) embebido en una página CSS-inline, → WeasyPrint. **Cuidado**: el CSS lleva `{}`, por eso el cuerpo se inyecta con `.replace("__BODY__", html)` y no con `.format()`. Si WeasyPrint falta o falla → `None` (el flujo continúa sin PDF).

4.10 Plantilla `report_template.md.j2`

Secciones del informe generado:

1. **Introducción** — cadena de confianza.
2. **Resumen ejecutivo** — tabla por severidad (●●●●) + total.
3. **Alcance** — tabla de activos evaluados.
4. **Detalle técnico** — por activo: tabla de ejecuciones (comando/estado/duración) y por hallazgo: severidad, comando relacionado, evidencia en crudo (bloque `text`), descripción, remediación y CVEs.
5. **Sugerencias CVE** — las obtenidas por enriquecimiento NVD (o aviso de no consulta).
6. **Apéndice** — salida completa de cada run (excerpt).
7. **Integridad** — SHA-256 + instrucción de verificación con `verify.py`.

5. La API: `api/`

5.1 `main.py` — bootstrap

- `lifespan` llama `init_db()` al arrancar y logea el modo de almacenamiento real.
- CORS restringido a los orígenes del propio panel (`127.0.0.1:<port>` , `localhost:8002`).
- Monta `ui/` como `StaticFiles` en `/static` , sirve `index.html` en `/` .
- `GET /health` → `{status, storage ("postgres"|"sqlite"), db_up, tools:{nmap:bool,...}}` con sonda SQL real en cada llamada. Es el corazón del botón/status del panel.

5.2 `routes/deps.py` — dependencias compartidas

- `get_db()` : abre sesión por petición; captura `ValidationError` / `ToolUnavailable` → **HTTP 400** con el mensaje; rollback + re-raise para el resto; cierra siempre.
- `find_or_404()` , `http_400()` , `http_404()` — helpers.

5.3 Endpoints

TARGETS (`API/ROUTES/TARGETS.PY`) — PREFIJO `/TARGETS`

Método	Ruta	Body/Query	Respuesta
GET	<code>/targets</code>	—	<code>{targets: [{id,ip,hostname,os,description,created_at,run_count,fi</code>
POST	<code>/targets</code>	<code>{ip*,hostname?,os?,description?}</code>	201 + payload
GET	<code>/targets/{id}</code>	—	payload o 404
DELETE	<code>/targets/{id}</code>	—	204 (borra historial en cascada)

SCANS (API/ROUTES/SCANS.PY) — PREFIJO /SCANS

Método	Ruta	Body	Respuesta
GET	/scans/ tools	—	catálogo <code>module_public()</code> : slug, name, tool, category, desc, available (PATH real), raw, params
POST	/scans	{target_id, module_slug, params}	201 + run payload + <code>output</code> completo
GET	/scans	—	últimos 100 runs, orden inverso
GET	/scans/ {id}	—	run + <code>output</code> + <code>findings</code> (resumen)

El endpoint `POST /scans` es **síncrono**: espera a que la herramienta termine (hasta el timeout) y devuelve el run finalizado. Simple y suficiente para un panel local.

FINDINGS (API/ROUTES/FINDINGS.PY) — PREFIJO /FINDINGS

Método	Ruta	Body	Respuesta
POST	/findings	{run_id*, severity*, title*, description?, remediation?, evidence_text?, note?, lookup_cves?}	201 + payload (con cves y evidencias)
GET	/findings	—	join con run/target: {findings: [{..., target, command, tool_name}]}
POST	/findings/ cves/suggest	{keyword} (embebido)	{cves:[...]} desde NVD

`_write_evidence()` escribe `data/evidence/finding-<id>.txt` ; el hallazgo se crea con `db.flush()` **antes** para poder numerar el fichero con su id. `_suggest_cves()` corta la keyword a 200 caracteres y la guarda como CSV en `cve_ids`.

REPORTS (API/ROUTES/REPORTS.PY) — PREFIJO /REPORTS

Método	Ruta	Body/Query	Respuesta
POST	/reports	{target_ids?, enrich_cves?, sign? =true, pdf?=true}	{md:{...}, pdf:{...}, signature:{...}, nvd_enrichment}
GET	/reports	—	lista de ficheros {name,size,kind} por mtime
GET	/reports/{file}	—	descarga (.md / .pdf / .sig)
GET	/reports/{file}/ verify	—	<code>verify_file()</code> → {ok,...}

`_safe_path()` **bloquea traversal**: el nombre no puede contener `/` ni `\`, y solo admite extensiones `.md|.pdf|.sig`. Un `400` ante nombres raros.

6. El panel: `ui/`

6.1 `index.html` — estructura de la SPA

Un único documento con 5 vistas (`section.view`) y navegación por botones:

Activos · Ejecutar · Resultados · Hallazgos · Reportes

- **Topbar** con la marca VulnFlow y 3 **LEDs** (`API` / `BD` / `Herramientas`) que se pintan en verde o rojo según `/health` .
- **Dialog de marcado** (`<dialog>`) que captura severidad, título, descripción, remediación, evidencia (texto seleccionado) y flag de búsqueda CVE.
- Assets referenciados con **cache-buster** `?v=20260925b` (editar estáticos → subir `?v=`).

6.2 `app.js` — lógica (vanilla JS, ~310 líneas)

Bloque	Función
<code>api()</code>	wrapper fetch JSON con manejo de <code>204</code> y extracción de <code>detail</code>
<code>switchView</code>	navegación por clases <code>active</code>
<code>refreshHealth</code>	LED state cada 5 s
<code>loadTargets</code> / <code>delTarget</code>	tabla de activos + borrado con confirmación
<code>fillTargetSelects</code>	rellena los <code><select></code> de run y reporte
<code>loadTools</code> / <code>renderParams</code>	catálogo agrupado por categoría; módulos sin herramienta quedan disabled ; dibuja los campos de params
<code>openRun</code>	muestra el run con su salida en <code>textarea</code>
marcado	<code>mark-sel-btn</code> lee <code>selectionStart/End</code> del textarea y abre el diálogo con esa evidencia; <code>suggestCves</code> llama a NVD en vivo
<code>loadRuns</code> / <code>viewRun</code>	historial
<code>loadFindings</code>	tabla con badges por severidad
<code>loadReports</code> / <code>verifyReport</code>	lista, descarga y verificación desde la UI
<code>boot</code>	carga todo en paralelo

Detalle del **marcado**: el `textarea` usa la selección nativa del navegador (`selectionStart:selectionEnd`) — sin rangos de autoría propios, sin dependencias.

6.3 `style.css` — tema

Tokens en `:root` (`--bg:#0f172a` , `--surface:#1e293b` , `--accent:#10b981` , `--error:#ef4444` , `--text:#e2e8f0` , ...). Fuentes vendidas vía `@font-face` local.
Badges por severidad con translucidos; LED con glow; responsive (sidebar 64px en móvil).

7. Infraestructura y scripts

Script	Funcion
<code>instalar.sh</code>	<code>python3 -m venv .venv + pip install -r requirements.txt</code>
<code>arrancar.sh</code>	<code>start</code> (uvicorn en background, pid <code>.api.pid</code> , log <code>.api.log</code> , espera a <code>/health</code>), <code>stop</code> , <code>status</code> , <code>log</code>
<code>infra/levantar-db.sh</code>	wrapper de <code>docker compose -f infra/docker-compose.yml ...</code> ; si <code>docker version</code> falla, reenvía a <code>sg docker -c "docker ..."</code> (grupo docker no activo en shells viejas)
<code>seed.py</code>	demo: crea target <code>127.0.0.1</code> y ejecuta <code>dig_dns</code> + <code>curl_http</code>
<code>verify.py</code>	CLI de verificación de firma (<code>.venv/bin/python verify.py data/reports/report_*.md</code>)

`infra/docker-compose.yml`: `postgres:16`, contenedor `vulnflow-db`, `5434:5432`, volumen `vulnflow_pgdata`, `healthcheck` con `pg_isready`. Credenciales demo `vulnflow/vulnflow/vulnflow` (solo local).

`requirements.txt`: pins `≥` (probado en Python 3.14; los pins estrictos rompían).

8. Seguridad por diseño (resumen de superficies)

Superficie	Mecanismo
Inyección de comandos	<code>argv</code> en lista, catálogo cerrado, parámetros extra ignorados, validadores <code>v_*</code>
Módulos raw	<code>VULNFLOW_ALLOW_RAW=1</code> + validación solo por <code>builder</code> (y aviso en UI)
Procesos colgados	<code>start_new_session</code> + <code>killpg</code> en timeout (SIGTERM→SIGKILL)
Path traversal	<code>_safe_path()</code> en <code>/reports/{file}</code> ; rutas de evidencia siempre relativas internas
Traversal por parámetros	<code>v_wordlist</code> limita a <code>.txt</code> alfanuméricos rutados
CORS	solo orígenes del propio panel
Integridad del informe	SHA-256 incrustado + firma Ed25519 cubriendo los bytes finales
BD caída	degradación a SQLite, <code>/health</code> reporta <code>db_up:false</code> , <code>storage</code> real
Internet/NVD caída	cache + rate-limit + <code>[]</code> silencioso

9. Dónde se guarda qué (`data/`, `gitignored`)

```
data/
├── evidence/
│   ├── run_<id>/output.txt      salida completa (stdout+stderr)
│   ├── run_<id>/output.xml      salida XML de nmap (si procede)
│   └── finding_<id>.txt         evidencia marcada del hallazgo
├── reports/
│   ├── report_<prefix>_<ts>.md  informe técnico
│   ├── report_<prefix>_<ts>.pdf exportación PDF
│   └── report_<prefix>_<ts>.sig firma Ed25519 (cubre el .md final)
├── keys/
│   ├── signer_ed25519.key       clave privada (0600)
│   └── signer_ed25519.pub       clave pública
└── vulnflow.db                 fallback SQLite (si Postgres no existe)
```

10. Cómo extenderlo

Añadir una herramienta al catálogo (el caso común):

1. Escribe un `builder(target, params) → list[str]` en `core/executor.py`.
2. Registra un `Module` con su `slug`, categoría, descripción y `Params` (con sus validadores).
3. La API (`/scans/tools`), el panel (menú agrupado + params dinámicos) y el informe (comando citado) lo exponen **solos**. No hay que tocar ninguna ruta.

Otras extensiones naturales: nuevo tipo de evidencia (`image|file` ya está en el modelo), verificación de firmas con clave pública externa, colas asíncronas para runs largos, cobertura por `cve_ids` en el resumen, etc.

11. Índice de ficheros (mapa rápido)

core/config.py	configuración + directorios + dump()
core/models.py	SQLAlchemy: Target, ToolRun, Finding, Evidence
core/session.py	engine con fallback Postgres-SQLite
core/executor.py	catálogo MODULES, validadores, builders, execute()
core/parser.py	parseo nmap XML + hints de versión
core/nvd.py	cliente NVD (cache + rate-limit + degradación)
core/signer.py	Ed25519: sign_file / verify_file
core/reporter.py	contexto → Jinja2 → MD → PDF + sign_report
api/main.py	FastAPI, middleware, /health, /, /static
api/routes/deps.py	get_db, http_400/404, find_or_404
api/routes/targets.py	CRUD de activos
api/routes/scans.py	catálogo + ejecución + historial
api/routes/findings.py	crear hallazgo (marcado) + sugerencia CVE
api/routes/reports.py	generar/descargar/verificar reportes
templates/report_template.md.j2	la plantilla del informe
ui/index.html	SPA (5 vistas + dialog de marcado)
ui/static/app.js	lógica vanilla (fetch, marcado, boot)
ui/static/style.css	tema dark slate/esmeralda, offline
ui/static/fonts/	Inter + JetBrains Mono (ventas, SIL OFL)
ui/static/favicon.svg	icono
infra/docker-compose.yml	PostgreSQL 16 :5434
infra/levantar-db.sh	wrapper docker (+ sg docker -c)
instalar.sh · arrancar.sh	· seed.py · verify.py
README.md · AGENTS.md	entrada y mapa mental del repo

12. Errores, fallos y degradaciones (behavior bajo estrés)

Escenario	Comportamiento
Postgres caído al arrancar	API sigue, /health → storage:sqlite, db_up:false
Postgres cae después de arrancar con engine fijado	Error de conexión en peticiones (buen motivo para reiniciar el agente para re-probar)
Herramienta no instalada	Módulo disabled en UI + 400 claro en API
Módulo desconocido	400 «Módulo desconocido»
Run en timeout	status=timeout, grupo matado, salida parcial conservada
NVD offline	lookup_cves /enriquecedor → []; el informe avisa «no se consultaron CVEs»
WeasyPrint ausente/roto	pdf:null; Markdown y firma intactos
Informe manipulado	verify → ok:false (firma no coincide)
Severidad malformada	400 desde la ruta (antes de insertar en el enum de Postgres)
Path traversal en /reports/{file}	400 (nombre no válido)

Este documento describe el estado del código en el momento de redactarse (VulnFlow 1.0.0). Si algo se desvía de lo escrito, la fuente de verdad es el propio código y el `AGENTS.md` del repo.